Decentralized Chaos to Centralized Order: An AI-Assisted Architecture for Shared-Faculty University Timetable Management

Abstract

University timetable management requires coordinating teachers, courses, semesters, sections, and timetable slots across departments. When each department is responsible for creating timetables independently, faculty members who teach across several departments may be assigned to simultaneous classes. This paper presents an AI-powered timetable management system that provides centralized coordination of departmental timetables. The platform follows a role-based workflow involving administrators, Deans, and Chairmen, and timetable approval follows a structured process. Rule-based validation identifies teacher and batch/section conflicts during timetable creation. For timetable retrieval, the system uses the Google Gemini API to interpret natural-language queries and extract intent and entities such as department, academic year, semester, batch, section, and teacher name. The backend then executes deterministic queries against a MongoDB database. This keeps query execution within the application layer and limits the database operations available to the language model, although it does not eliminate the risks of incorrect or manipulated interpretations. The system's features were demonstrated through representative application workflows. This functional validation was not a user study or a quantitative performance evaluation. The system demonstrates how AI can serve as an assistance layer within a conventional timetable management architecture without autonomously generating or optimizing schedules.

Keywords: university timetabling, conflict validation, role-based access control, natural-language queries, MERN stack

1. Introduction

University timetable management is an essetial academic and faculty task that directly affects the organization of teaching and learning. A timetable is a combination of mutltiple elements such as teachers, courses, semesters, academic years and available slots. As the number of departments, courses and teaching staff increases, preparing and managing timetables becomes diffcult, specially when the same faculty is taching in the multiple departments. In many universities environments, timetable management is handled by individual departments. Although departmental management can simplify local scheduling, it can create coordination

challenges when academic resources are shared across departments. A teacher can be assigned to multiple departments at the same time if the departmenttal timetables are prepared without centralized coordination. Identifying, and resolving such conflicts manually requires repeated communication between departments and may increase the faculty effort involved in timetable creation.

The problem becomes more significant when timetable information is distributed across separate administrative processes. Different academic authorities may be responsible for managing departments, teachers, courses, and timetable approvals, making it necessary to maintain a structured workflow for creating, reviewing, and approving timetables. A centralized system can provide a common platform through which these activities can be coordinated while preserving appropriate responsibilities for different administrative roles.

Recent advances in artificial intelligence (AI) have also introduced new possibilities for interacting with information systems through natural language. An AI-assisted interface can allow users to express information requests in natural language, while the system interprets relevant entities and user intent from the request (Quamar et al., 2022). However, integrating AI into an information management system requires a clear separation between language interpretation and actual data operations so that database access remains under application-level control.

This paper presents an AI-assisted university timetable management system that combines centralized timetable administration with role-based access control, rule-based conflict validation, timetable approval workflows, and natural-language information retrieval. The system is implemented using the MERN stack, with React used for the frontend, Node.js and Express.js for backend services, and MongoDB for data management. The AI-assisted query component uses the Google Gemini API to interpret natural-language timetable queries and extract relevant entities. The backend subsequently uses these structured results to perform deterministic database queries and return the requested information.

The primary focus of the proposed system is not autonomous timetable generation or schedule optimization. Instead, it aims to provide a centralized environment for managing university timetable information while using AI as an assistance layer for querying that information. By combining conventional rule-based application logic with AI-assisted natural-language interaction, the system provides an architecture for managing shared-faculty scheduling

challenges while maintaining explicit control over timetable data and administrative workflows.

2. Problem Statement

University timetable preparation becomes increasingly challenging when academic departments prepare their schedules independently while relying on shared teaching staff. University timetabling research has examined complex, real-world scheduling requirements across different institutions (Müller et al., 2025). In a decentralized scheduling environment, departments may prepare timetables according to their individual course, section, and faculty requirements. However, when a teacher serves multiple departments, a lack of coordination can result in the teacher being assigned to different classes during the same time slot. Dimopoulou and Miliotis (2004) describe a university timetabling system that coordinates departmental data through a shared database and addresses conflicts associated with distributed timetable construction. These challenges motivate the need for centralized coordination and conflict validation across departmental timetables.

Furthermore, timetable information may need to pass through multiple academic authorities before it is finalized, creating a need for a structured process for timetable creation, review, approval, and publication. Another challenge is the way users interact with timetable information. Conventional timetable systems generally require users to navigate predefined forms, filters, or dropdown menus to locate specific information. While these interfaces are useful for structured data entry and management, they may require multiple steps when users simply want to ask questions about teachers, sections, semesters, or departmental schedules.

Therefore, there is a need for a centralized timetable management platform that can support cross-department faculty coordination, validate relevant scheduling conflicts during timetable creation, provide controlled administrative workflows, and support more natural interaction with timetable information. The proposed system addresses these requirements by combining centralized management, role-based access control, rule-based conflict validation for teachers and batches/sections, timetable approval workflows, and an AI-assisted natural-language query mechanism.

3. Research Objectives

The main objective of this study is to develop a centralized and AI-assisted university timetable management system capable of supporting timetable creation, coordination, validation, approval, and information retrieval. The specific objectives are:

1. **Centralize Timetable Management:** To provide a common platform for managing departments, teachers, courses, semesters, sections, and timetable slots.
2. **Reduce Shared-Faculty Scheduling Conflicts:** To validate teacher availability during timetable slot creation and prevent conflicting teacher assignments.
3. **Manage Batch and Section Conflicts:** To validate timetable assignments against existing section and batch schedules.
4. **Implement Role-Based Access Control (RBAC):** To separate administrative and academic responsibilities among Super Administrators, Deans, and Chairmen.
5. **Establish Approval Workflows:** To implement a structured workflow where departmental timetables can be submitted for review and subsequently approved or rejected by the relevant academic authority.
6. **Provide AI-Assisted Querying:** To enable natural-language timetable information retrieval through an interface that allows users to query information without relying exclusively on predefined dropdowns and filters.
7. **Maintain Control over AI Operations:** To integrate natural-language interpretation with deterministic database operations by using the Gemini API to extract query intent and entities while allowing the backend to control the actual database queries.
8. **Develop Web Architecture:** To build a web-based system using modern full-stack technologies that provides a maintainable architecture for university timetable administration.

4. Related Work

University course timetabling has been extensively studied as a complex scheduling problem involving multiple hard and soft constraints. These constraints may include instructor availability, student requirements, time slot assignments, and resource limitations. Chen et al. describe the University Course Timetabling Problem as a challenging combinatorial optimization problem and review the different methodologies developed to address it (Chen et al., 2021).

Recent research continues to investigate increasingly realistic timetabling problems, including the integration of student sectioning with course time and room assignments in real-world university environments (Müller et al., 2025).

Several studies have focused on centralized and distributed approaches to university timetable management. Dimopoulou and Miliotis presented a university course timetabling system based on a centralized database in which different departments could access and update relevant information (Dimopoulou & Miliotis, 2004). Their system centrally planned common courses involving students from multiple departments and considered teacher availability when constructing departmental timetables. This work demonstrates the importance of sharing timetable-related information across departmental boundaries when academic resources are not isolated within individual departments.

Inter-department scheduling conflicts have also been investigated using computational and intelligent scheduling approaches. Nourmohammadi-Khiarak et al. proposed a multi-agent approach for controlling collisions involving common events between university departments (Nourmohammadi-Khiarak et al., 2020). Their approach considered the coordination of separately generated departmental schedules to reduce conflicts involving shared academic resources.

In addition to centralized coordination, existing research has investigated automated timetable construction and optimization. Existing timetabling approaches formulate the problem using methods such as constraint-based modeling, mathematical optimization, and heuristic or metaheuristic search (Chen et al., 2021). Such methods can be used to generate schedules under multiple constraints, while some studies specifically address coordination problems such as inter-department common-event collisions (Nourmohammadi-Khiarak et al., 2020). However, these approaches primarily address the computational construction, optimization, or coordination of timetable solutions. The proposed system addresses a different application-level focus by combining shared-faculty conflict validation with centralized administrative approval workflows and an AI-assisted natural-language query mechanism for retrieving timetable information. In particular, the system does not use AI to autonomously generate or optimize timetables; instead, Gemini is used as an interpretation layer, while timetable retrieval and database operations remain under backend application control.

Natural-language interaction with databases represents another relevant research direction. Recent surveys have examined the use of large language models for natural-language-to-

database query tasks, including the interpretation of user intent, schema understanding, and conversion of natural-language questions into structured queries (Hong et al., 2025; Liu et al., 2025; Zhu et al., 2024). These developments demonstrate the potential of language models to provide more accessible interfaces for database-driven applications. However, allowing a language model to directly control database operations introduces additional concerns regarding query correctness, schema interpretation, and controlled access to data (Hong et al., 2025; Liu et al., 2025; Zhu et al., 2024).

The proposed system applies this interpretation-and-execution pattern to university timetable management with a specific combination of features. Gemini is used primarily for entity and intent extraction from natural-language timetable questions rather than for autonomous timetable generation or unrestricted database execution. The extracted parameters are processed by the Node.js/Express backend, which validates the required parameters and verifies the corresponding department, semester, batch, and section entities before retrieving timetable information through MongoDB/Mongoose queries. The system further integrates this AI-assisted retrieval mechanism with centralized timetable management, shared-faculty conflict validation, and a multi-level timetable approval workflow. Thus, the contribution is not the interpretation-then-execution pattern itself, but its integration into a timetable management architecture in which the language model is restricted to query interpretation and database operations remain under application-level control. However, backend validation does not guarantee that the language model has correctly interpreted the user's request, and manipulated or ambiguous inputs may still influence the extracted entities or filters. Therefore, additional validation and protection against manipulated inputs remain important considerations for the reliability and security of the AI-assisted query mechanism (Pedro et al., 2025).

5. Proposed System

The proposed system is a centralized web-based university timetable management platform designed to coordinate timetable-related activities across academic departments. The system combines role-based administrative management, timetable creation, rule-based conflict validation, timetable approval, and AI-assisted natural-language information retrieval within a single application.

5.1 System Roles and Responsibilities

The proposed system defines multiple administrative roles to support the university timetable workflow:

- **Super Administrator:** Manages university-level administrative information. Responsible for the appointment and management of Deans, oversight of faculties, and assignment of Deans to their respective faculties. The Super Administrator can also access approved timetable information through the dashboard.
- **Dean:** Operates at the faculty level. Manages Chairmen and departments within the assigned faculty and is responsible for reviewing submitted timetables (approving or rejecting them).
- **Chairman:** Responsible for departmental academic and timetable management. Manages teachers, sections/batches, semesters, courses, and timetable slots within the department. The Chairman creates departmental timetables while the system validates relevant scheduling conflicts during slot assignment.
- **Guest/Student:** Interacts with the system to view timetable information through structured selections or the AI-assisted query interface.

The role separation follows the general RBAC principle of assigning permissions according to defined user roles and responsibilities rather than granting the same permissions to all users (Ferraiolo et al., 1999).

5.2 Centralized Timetable Management and Conflict Validation

Academic information such as departments, teachers, courses, semesters, sections, and timetable slots is maintained within a common database. When a Chairman creates a timetable slot, the backend validates the assignment against existing timetable information. This mechanism particularly addresses shared-faculty scheduling: if a teacher has already been assigned to another class during the selected time period (even in a different department), the system prevents the conflicting assignment.

5.3 Timetable Approval Workflow

Timetable creation and approval are treated as separate stages within the system. After a Chairman prepares a departmental timetable, it is submitted for review by the relevant Dean. The Dean can inspect the details and either approve or reject the submission. Approved timetables become publicly viewable across the institution.

5.4 AI-Assisted Query Architecture

The system incorporates an AI-assisted query mechanism to provide a natural-language interface for timetable information. Rather than allowing the language model to directly control database operations, the architecture separates natural-language interpretation from data retrieval. When a user submits a natural-language question, the query is sent to the AI processing component. The Google Gemini API is used to interpret the query and extract structured information relevant to timetable retrieval. Depending on the request, extracted information may include the department, academic year, semester, section, or teacher name.

The resulting structured representation is then processed by the backend. The backend determines the appropriate database operation and executes the corresponding query against MongoDB using application-level logic and Mongoose. The retrieved information is subsequently returned to the user through the application interface. The language model is restricted to interpreting the user's natural-language query and extracting structured parameters; it does not directly execute database operations. This design positions Gemini as an interpretation component rather than a direct database controller, while database access remains within the backend application logic. However, this separation does not eliminate risks in the interpretation layer, as manipulated or ambiguous queries may result in incorrect entities or filter values. Therefore, the current architecture provides controlled backend execution but does not guarantee the correctness or security of the language-model interpretation, leaving residual risk from prompt-injection and other manipulation techniques (Pedro et al., 2025).

This design means that Gemini serves as an interpretation component rather than a direct database controller. Database access remains within the backend application logic.

6. System Architecture and Workflow

The proposed system follows a layered client-server architecture in which the presentation layer, application layer, data layer, and AI-assisted query component perform separate responsibilities.

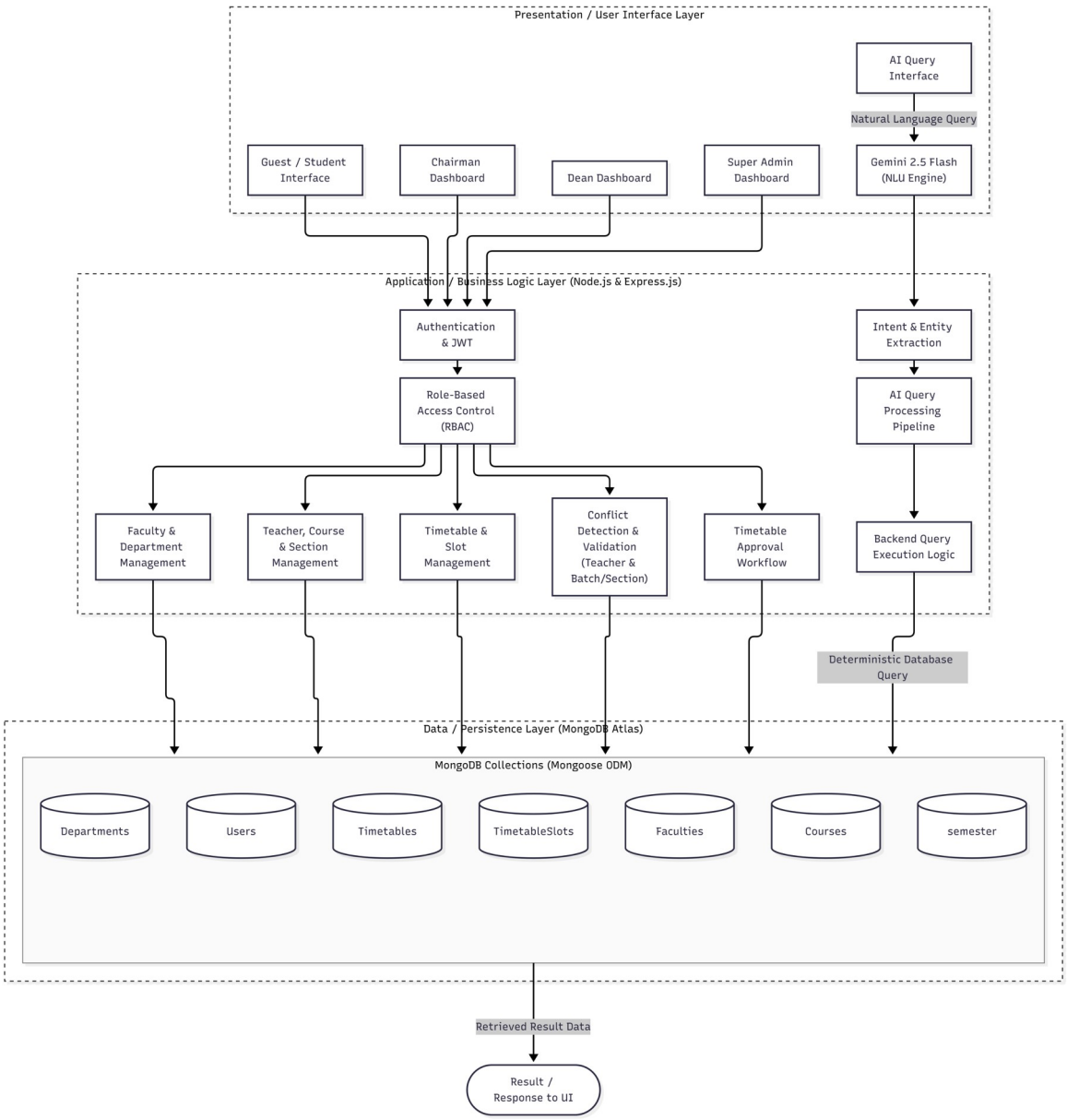


Fig. 1. Three-Level System Architecture of the proposed timetable management system showing the Presentation/User Interface, Application/Business Logic, and Data/Persistence layers.

The presentation layer contains the user-facing interfaces, including Guest/User timetable access, Chairman Dashboard, Dean Dashboard, Super Administrator Dashboard, and the AI Query Interface.

The application layer contains authentication, authorization, faculty and department management, teacher/course/section management, timetable and slot management, conflict validation, timetable approval, and AI query processing.

The data layer contains MongoDB Atlas and the primary application data, including users, faculties, departments, teachers, courses, sections/batches, timetables, and timetable slots.

The AI query component interacts with the application layer rather than directly accessing the database. Natural-language input is interpreted by Gemini, after which the backend processes the extracted information and executes the corresponding read-only query against the database. The retrieved timetable information is then returned to the user through the application interface.

6.1 Complete Administrative Workflow

The creation and approval of university timetables follows a hierarchical administrative workflow.

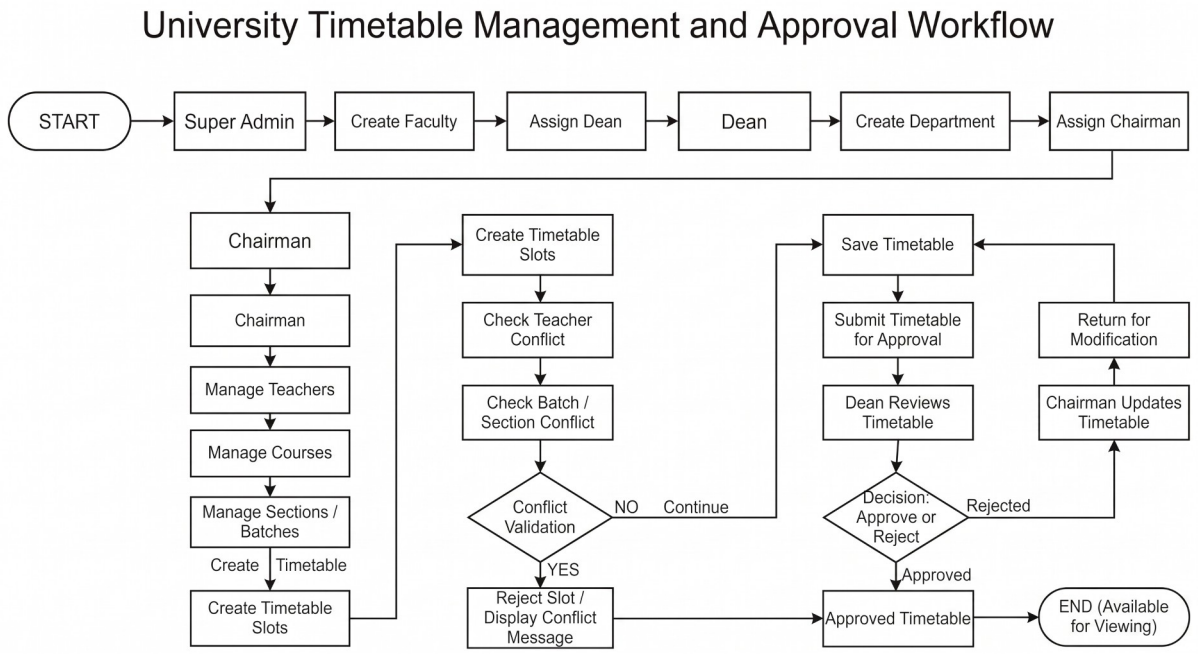


Fig. 2. Complete university timetable management and approval workflow from faculty and department configuration to timetable creation, conflict validation, and Dean approval.

The workflow begins with the Super Administrator creating faculties and assigning Deans. The Dean then manages departments and assigns Chairmen. After assignment, the Chairman manages departmental teachers, courses, sections/batches, and timetable information.

During timetable creation, each slot is passed through the conflict-validation mechanism. If a scheduling conflict is detected, the slot is rejected and the user is required to modify the assignment. If no relevant conflict is detected, the slot is saved and the timetable can subsequently be submitted for approval.

The Dean reviews the submitted timetable. An approved timetable becomes available through the timetable interface, while a rejected timetable is returned to the Chairman for modification.

6.2 Timetable Slot Conflict Detection and Validation

During slot allocation by a department Chairman, rule-based validation is performed deterministically.

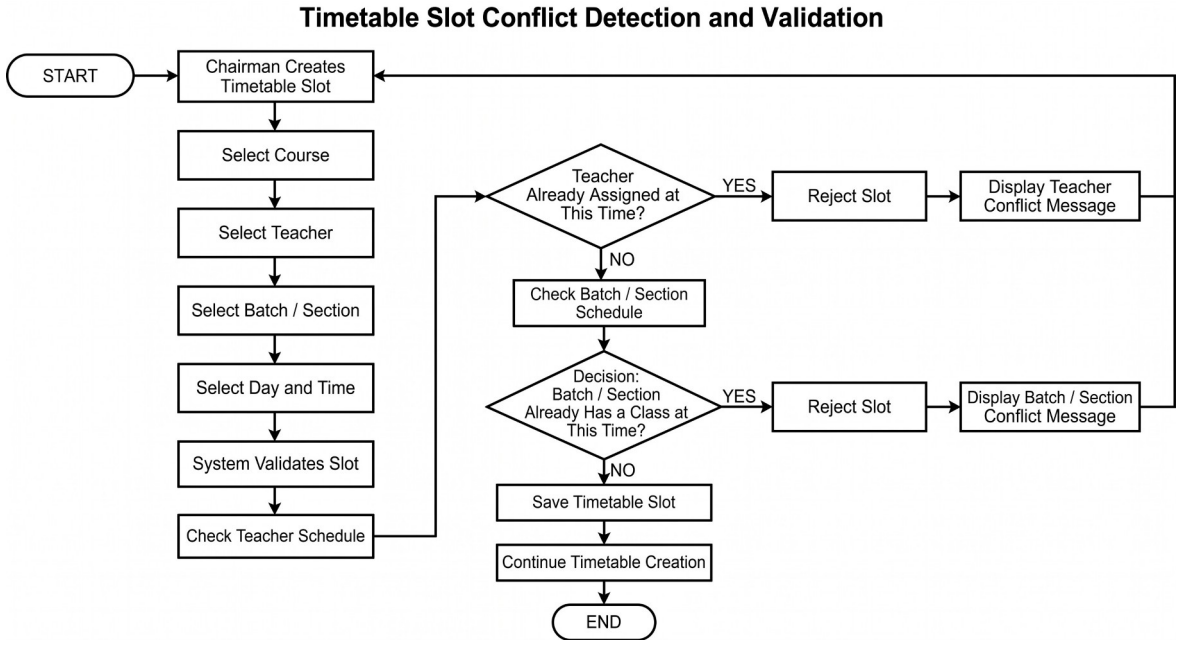


Fig. 3. Timetable slot conflict detection and validation flowchart for teacher and batch/section scheduling constraints.

The validation process begins when a Chairman creates a timetable entry by selecting a course, teacher, batch or section, and the corresponding day and time slot. Because preventing shared-teacher conflicts is a primary purpose of the system, the first validation check is performed against the selected teacher. The system searches the existing timetable assignments for that teacher and compares the scheduled day and time slot with the new assignment. A teacher conflict is identified when the same teacher is already assigned to another class for the same day and time slot, and the new assignment is rejected. If the teacher has no assignment for that day and time slot, the timetable entry can proceed to the subsequent batch/section conflict check. The current implementation uses predefined timetable slots and does not perform arbitrary time-interval overlap or recurring-event analysis.

If the system detects a teacher conflict, it rejects the slot and displays an error response. If no teacher conflict exists, the system continues to check the relevant batch or section schedule. If the batch or section is already occupied during the selected time, the slot is rejected. Otherwise, the slot is saved.

The current validation mechanism therefore focuses on two implemented constraints: teacher scheduling conflicts and batch/section scheduling conflicts. Classroom availability is not included in the current implementation.

6.3 AI-Assisted Query Pipeline

The AI-assisted query subsystem separates natural-language interpretation from database operations. Gemini is responsible for interpreting the user's query and extracting structured parameters, while the backend validates the required parameters and verifies the corresponding department, semester, batch, and section entities before retrieving timetable information. The backend then executes the corresponding read-only database query using application-level logic and Mongoose, and the language model does not directly access or modify the database. This separation bounds the database operations available to the language model, but it does not eliminate risks in the interpretation layer, since manipulated or ambiguous input may still result in incorrect entities or filter values. Therefore, the architecture provides controlled and validated backend execution without guaranteeing the correctness or security of the language-model interpretation (Pedro et al., 2025).

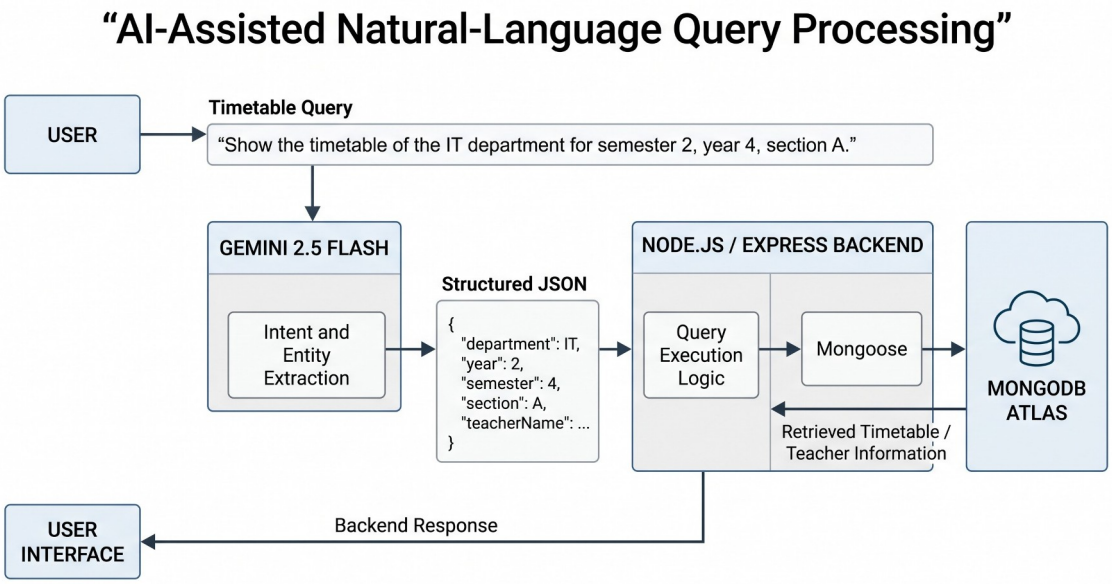


Fig. 4. AI-assisted natural-language query processing architecture showing the separation between Gemini-based interpretation and backend database operations.

A user can submit a natural-language query such as:

"Show IT department year 4 semester 2 section A timetable."

The Gemini 2.5 Flash model interprets the query and extracts relevant entities. An example structured representation may contain:

```
{
    "department": "IT",
    "year": 4,
    "semester": 2,
    "section": "A",
    "teacherName": null
}
```

The structured information is then passed to the Node.js/Express backend. The backend determines the appropriate query execution logic and uses Mongoose to retrieve the relevant information from MongoDB Atlas.

The retrieved information is returned through the backend response and displayed through the application interface. Gemini therefore functions as an interpretation layer rather than directly modifying or querying the database. The extracted structured parameters are validated by the backend and checked against the authenticated user's permissions before the corresponding read-only query is executed. This deterministic backend execution bounds the database access available to the language model but does not eliminate the risk of incorrect or manipulated entity values. Consequently, crafted or ambiguous natural-language inputs may still influence the extracted parameters, representing a residual prompt-injection and interpretation risk (Pedro et al., 2025).

7. Results and System Validation

Functional validation was conducted using representative application workflows covering the major implemented features. For each workflow, a representative input was provided, the expected functional outcome was identified, and the observed system behavior was compared with that expected outcome. For example, teacher and batch/section conflict scenarios were expected to be rejected when conflicting assignments were detected; timetable submissions were expected to proceed through the defined approval workflow; and representative natural-language timetable queries were expected to return the corresponding timetable information. The observed outcomes matched the expected functional behavior for these demonstrated workflows. These checks demonstrate that the major intended workflows and features are implemented and operational, while the evaluation does not constitute a quantitative performance assessment or user study.

7.1 Administrative Management

The Super Administrator provides university-level management of faculties and Deans. This dashboard offers access to the administrative operations required to establish the faculty-level structure of the system.

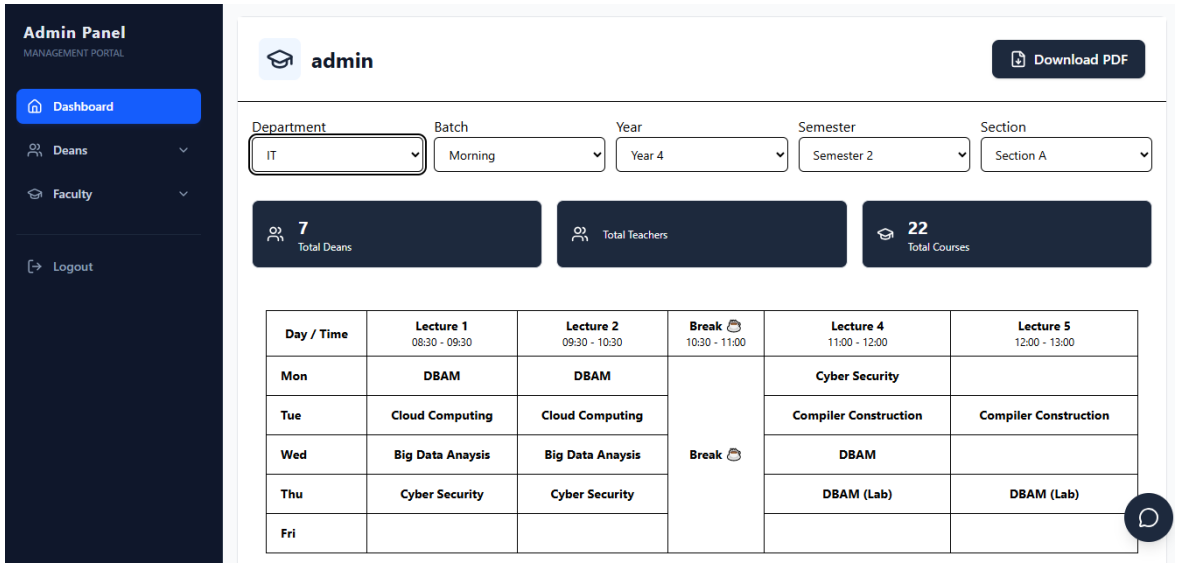


Fig. 5. Super Administrator dashboard for managing faculties and Deans.

The Dean interface provides faculty-level management functions, including department and Chairman management. This creates the administrative relationship required for subsequent timetable submission and approval.

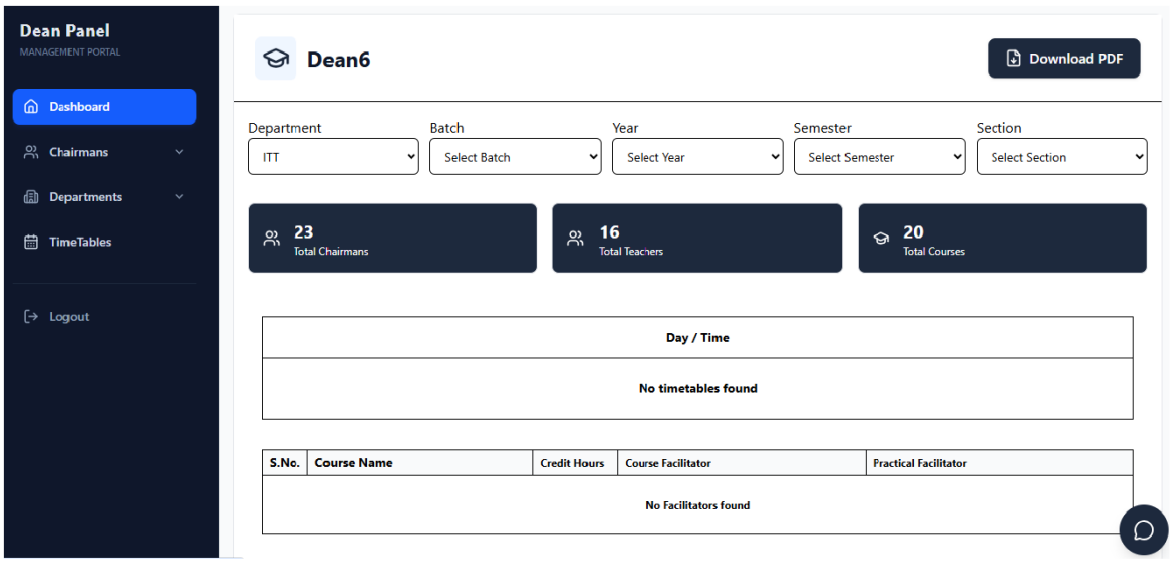


Fig. 6. Dean interface for managing departments and assigning Chairmen.

These interfaces demonstrate the separation of responsibilities between university-level and faculty-level administrative operations.

7.2 Departmental Timetable Management

The Chairman is responsible for managing departmental academic information and creating timetable schedules. The Chairman dashboard provides access to teacher, course, section/batch, semester, timetable, and slot management operations.

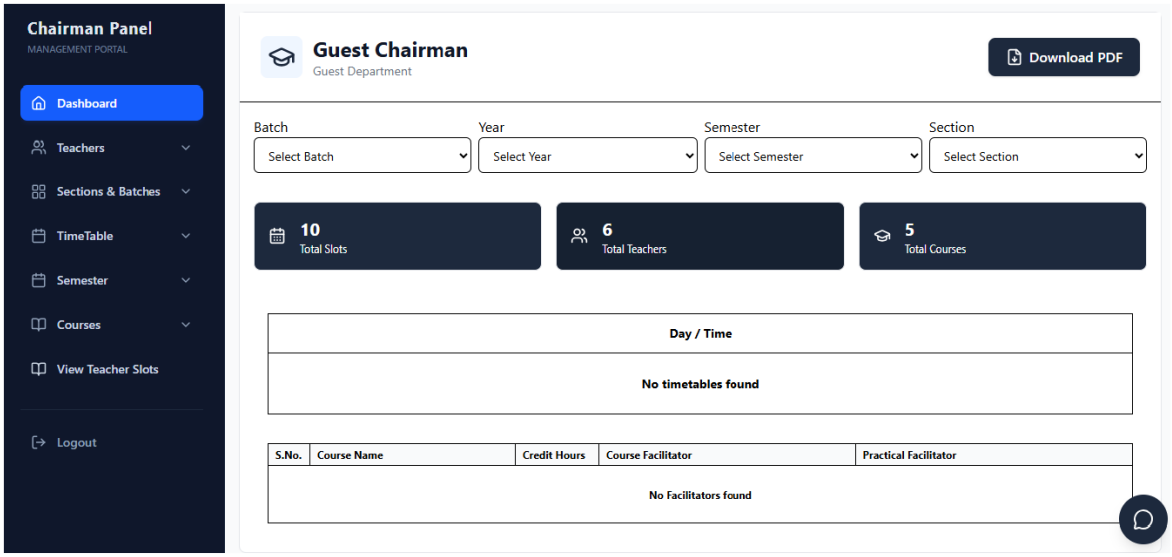


Fig. 7. Chairman dashboard for departmental academic and timetable management.

After the required academic information has been configured, the Chairman can create timetable slots by selecting the relevant course, teacher, section or batch, and time information.

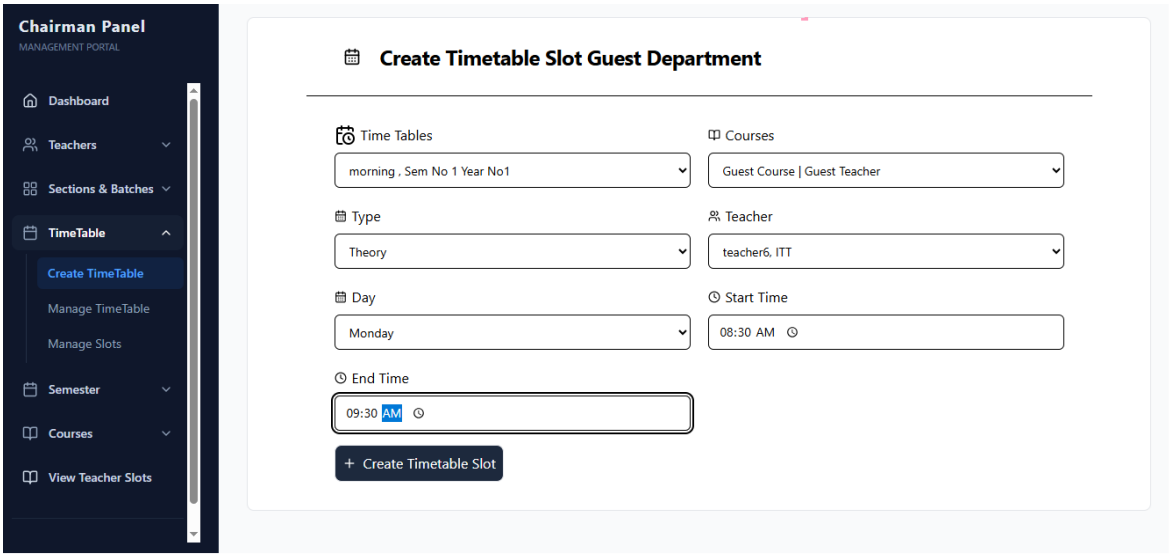


Fig. 8. Timetable slot creation interface used by the Chairman.

The slot-creation interface provides the entry point for the rule-based validation mechanism described in Section 6.2.

7.3 Conflict Detection and Validation

Conflict validation is one of the central functional features of the developed system. The implementation checks whether a selected teacher is already assigned to another class during the selected time period. When the system identifies such a conflict, the system prevents the creation of the conflicting slot.

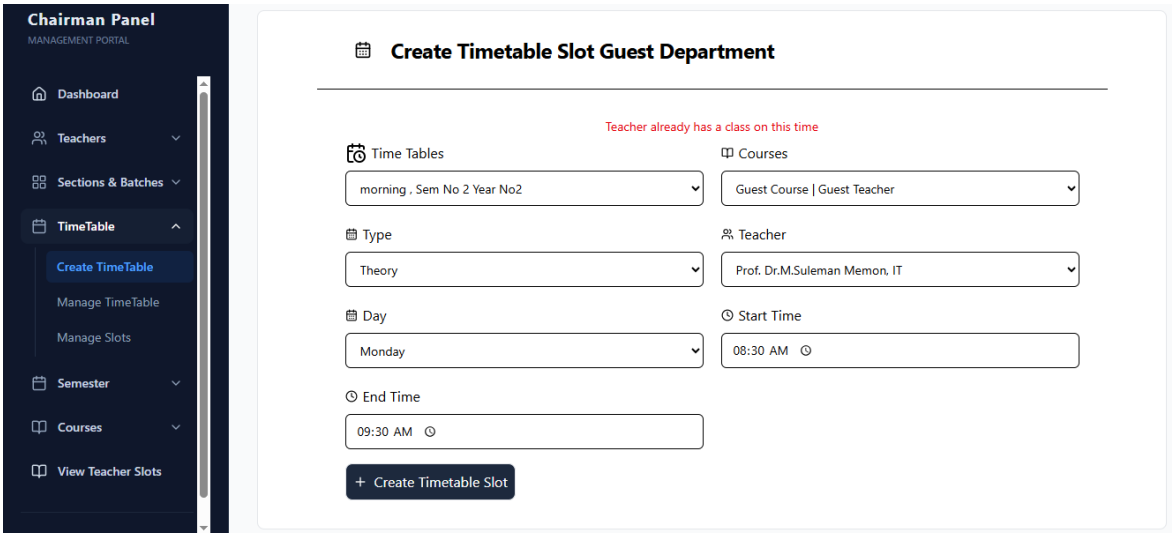


Fig. 9. Teacher scheduling conflict detected during timetable slot creation.

The conflict-validation functionality was evaluated using representative test cases during timetable creation. For teacher conflict validation, an existing timetable assignment was used for a teacher in a specific time slot, followed by an attempt to assign the same teacher to another class in the same slot. The system detected the conflict and prevented the conflicting assignment. For batch/section conflict validation, an existing assignment was used for a batch or section in a specific time slot, followed by an attempt to assign the same batch or section to another class during that slot. The system detected the conflict and prevented the conflicting assignment. As a non-conflicting control, assignments using different time slots were also tested, and the system allowed the assignment when no teacher or batch/section conflict was present. These results demonstrate that the implemented system performs rule-based conflict validation during timetable creation rather than relying solely on manual inspection after timetable preparation.

The current implementation validates teacher and batch/section scheduling conflicts but does not validate room or classroom conflicts. Therefore, room and classroom constraints are outside the scope of the reported results.

7.4 Timetable Approval Workflow

After timetable creation, the Chairman can submit the timetable for review. Submitted timetables become available to the relevant Dean through the pending timetable interface.

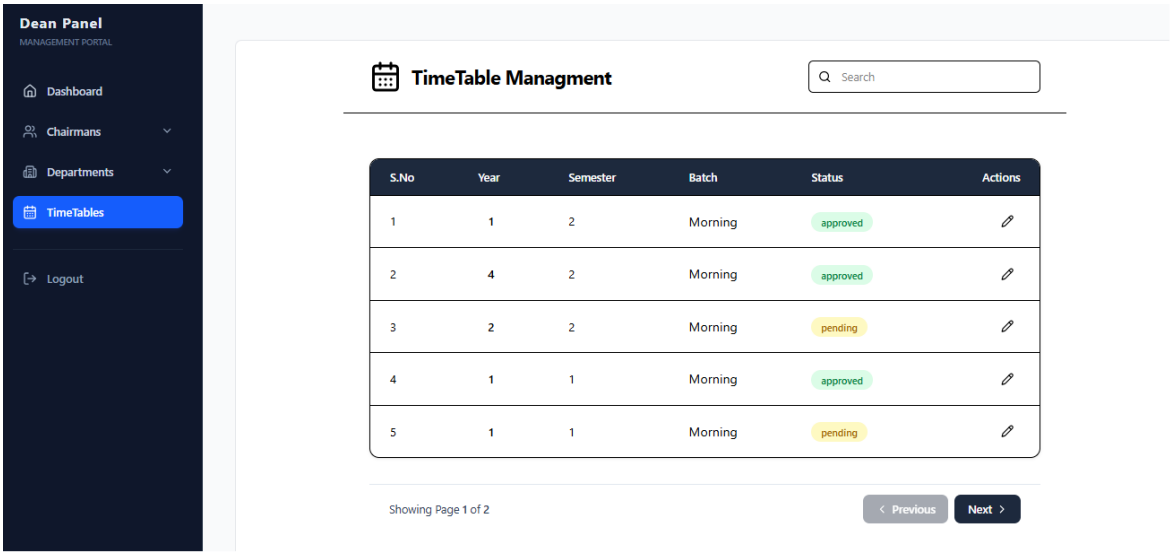


Fig. 10. Dean interface displaying submitted timetables pending approval.

The Dean can inspect the submitted timetable and make an approval or rejection decision.

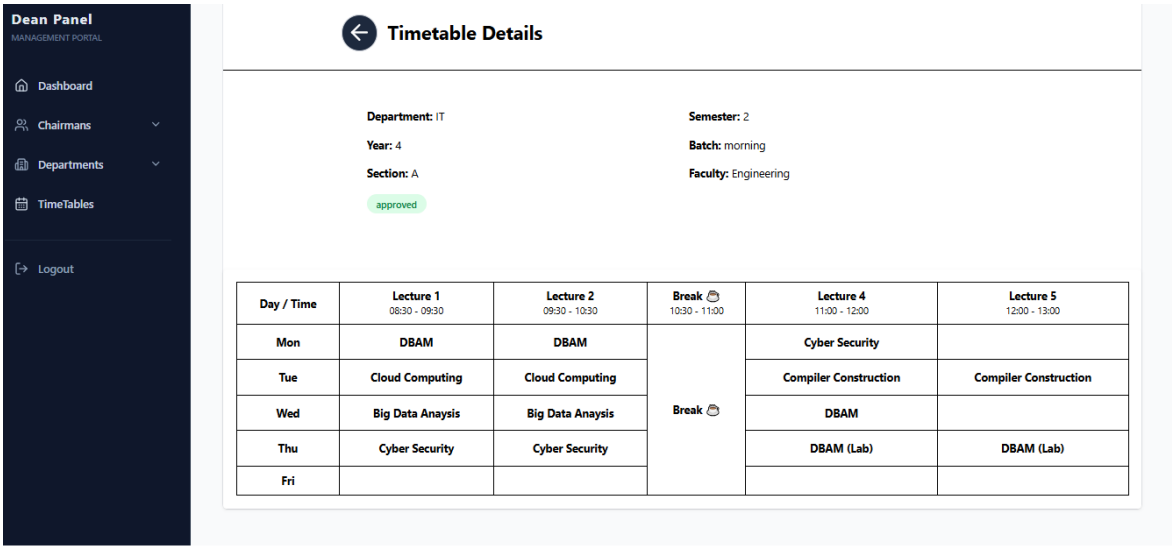


Fig. 11. Timetable review interface providing approval and rejection actions.

When a timetable is approved, it becomes accessible through the approved timetable interface. If it is rejected, the Chairman can modify the timetable and resubmit it. Functional validation of the workflow confirmed the relevant state transitions: a submitted timetable remained unavailable as an approved timetable until the Dean approved it, while a rejected timetable was returned to the Chairman for revision and resubmission. This workflow provides a clear separation between timetable creation and approval and adds an administrative review stage before timetable information is made available through the approved timetable interface.

7.5 AI-Assisted Natural-Language Query

The developed system provides an AI-assisted interface through which users can query timetable information using natural language. The user does not need to manually navigate through every timetable parameter when making a supported query.

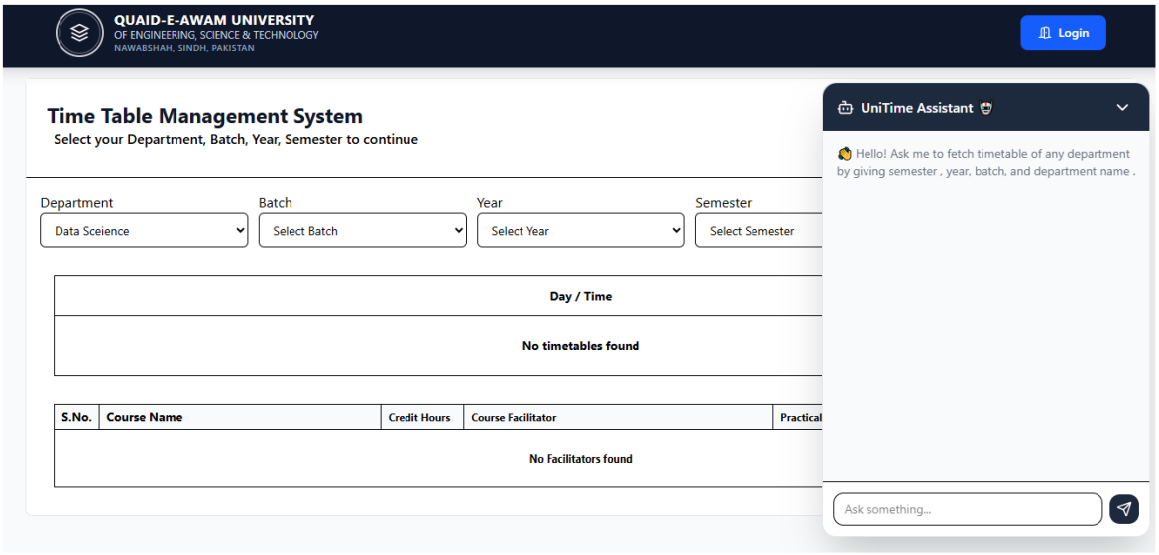


Fig. 12. AI-assisted natural-language query interface for timetable information retrieval.

For example, a user can submit a query specifying a department, year, semester, section, teacher, or batch. Gemini interprets the request and extracts the relevant entities. The backend subsequently uses those entities to perform a controlled database query.

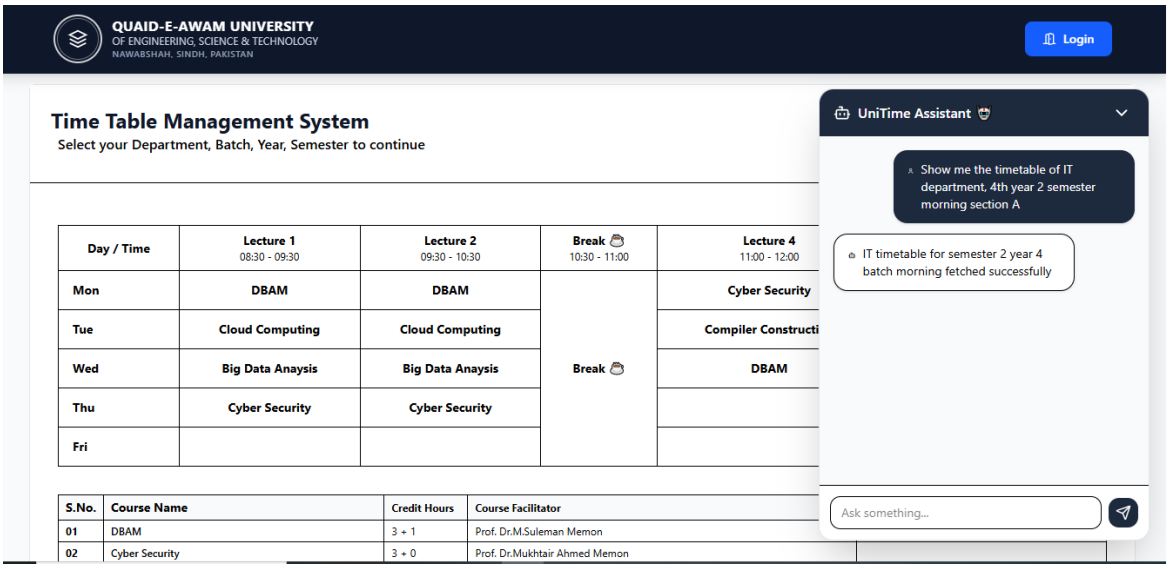


Fig. 13. Timetable information retrieved through the AI-assisted query mechanism.

The AI component provides an alternative natural-language interaction method for retrieving timetable information while leaving actual database operations under backend application control.

7.6 Timetable Viewing

The timetable viewing interface displays the final timetable to users. Users can access timetable information according to the available structured selections and approved timetable data.

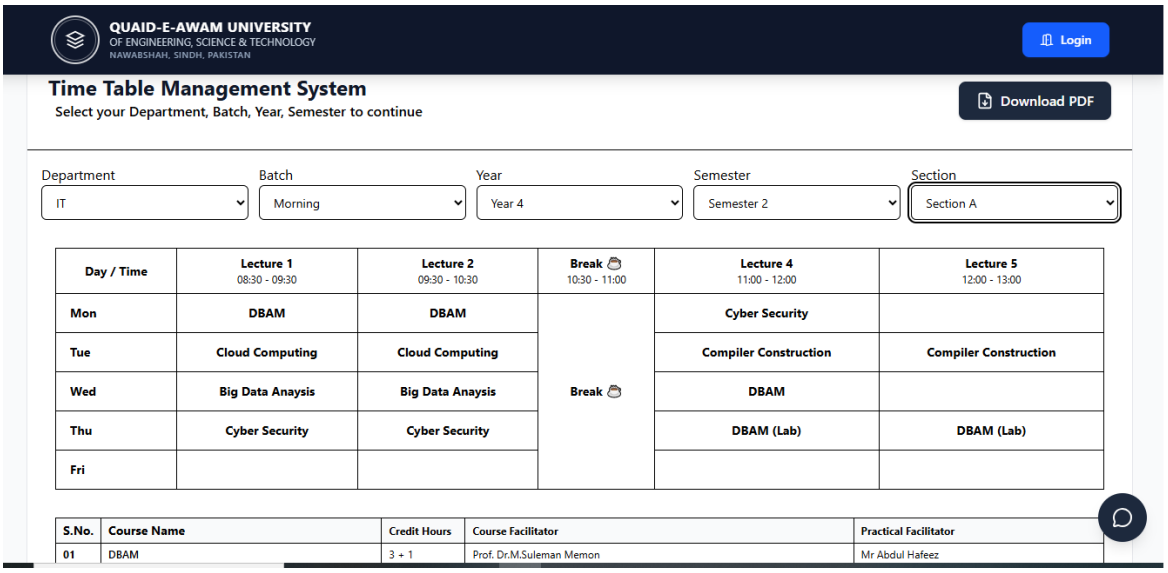


Fig. 14. Approved timetable displayed to the user through the timetable viewing interface.

The displayed timetable represents the final output of the implemented administrative workflow, from academic information management and slot creation to conflict validation and approval.

7.7 Summary of Functional Validation

The functional validation demonstrates the major capabilities implemented in the system, as summarized in Table 1.

Table 1. Summary of functional validation of the implemented system.

System Feature	Validation Result
Faculty and Dean management	Implemented and demonstrated
Department and Chairman management	Implemented and demonstrated
Teacher management	Implemented
Course management	Implemented
Section/Batch management	Implemented
Timetable and slot creation	Implemented and demonstrated
Teacher conflict validation	Implemented and demonstrated
Batch/Section conflict validation	Implemented and demonstrated
Room/Classroom conflict validation	Not implemented
Timetable submission	Implemented
Dean timetable review	Implemented and demonstrated
Timetable approval/rejection	Implemented and demonstrated
AI-assisted natural-language querying	Implemented and demonstrated
Autonomous timetable generation	Not implemented
Automated timetable optimization	Not implemented

The results indicate that the developed application provides the core centralized management, validation, administrative approval, and AI-assisted information retrieval capabilities defined in the research objectives. The current evaluation is functional rather than a large-scale quantitative assessment of usability, scalability, or optimization quality.

7.8 Functional Evaluation of AI-Assisted Timetable Retrieval

To determine the practical behavior of the AI-Assisted Natural-language query, I conduct functional evaluation using the deployed system. Total 34 natural-language queries were tested without modifying the code. The queries covered timetable retrieval with different wording patterns, incomplete requests, teacher schedule queries, and references to invalid or unavailable entities.

The evaluation focused on the system's observable responses rather than on the performance of the underlying language model in isolation. A query was considered successful when the system returned the requested timetable or teacher schedule. Other responses were categorized according to their observed behavior, including requests for missing parameters, unsupported entity recognition, authorization rejection, and entity-validation or retrieval failures.

Of the 34 tested queries, 13 returned the requested timetable or teacher schedule. Twelve prompted the user to supply one or more missing parameters. Three returned an unsupported-entity response, while one teacher-schedule query returned an authorization error and another

was not correctly interpreted by the system. Four additional queries resulted in entity-validation or retrieval failures involving a section, department/timetable, or semester reference.

These observations indicate that the query mechanism can retrieve timetable information across several natural-language formulations. In particular, seven timetable queries using different phrasings for the same department, semester, year, batch, and section returned the requested timetable. However, the tests also exposed cases in which the system did not correctly resolve a teacher reference or handle a supplied parameter. For example, a query containing “night” as the batch value prompted a request for the batch again, while a teacher-schedule request returned an authorization error during testing. These cases warrant further investigation to determine whether the underlying cause lies in query interpretation, entity mapping, validation logic, or access-control behavior.

The results should be interpreted as an initial functional assessment rather than a comprehensive measure of accuracy or reliability. The evaluation used a limited set of manually formulated queries and did not involve a controlled comparison of language models, a formal user study, or a large-scale performance benchmark. Nevertheless, it provides practical evidence of both successful retrieval behavior and failure cases in the deployed application, helping identify areas for future refinement.

Table 2. Observed outcomes of the 34 natural-language queries tested.

Observed outcome	**Queries**	**Percentage**
Requested timetable or teacher schedule retrieved	13	38.2%
Asked for missing parameters	12	35.3%
Unsupported entity response	3	8.8%
Entity-validation or retrieval failure	4	11.8%
Authorization error	1	2.9%
Query interpretation failure	1	2.9%

Percentages are rounded to one decimal place.

8. Limitations and Future Work

Although the proposed system provides centralized timetable management, conflict validation, role-based administration, and AI-assisted information retrieval, several limitations remain in the current implementation.

First, the conflict detection mechanism primarily addresses teacher and batch/section scheduling conflicts. The current version does not include room or classroom conflict validation. Future versions can extend the validation layer to include classroom availability, room capacity, equipment requirements, and other resource constraints.

Second, the system does not automatically generate or optimize complete timetables. Timetable creation remains a controlled administrative process in which authorized users assign courses, teachers, and time slots. Future research could investigate automated timetable generation and optimization techniques that consider multiple constraints simultaneously while retaining administrative control over the final schedule.

Third, the AI component is focused on natural-language query interpretation and information retrieval rather than autonomous timetable generation. Gemini is used to interpret user queries and extract relevant entities, while the backend performs the actual database operations. Future work could expand the conversational interface to support more complex multi-step queries, contextual follow-up questions, and richer timetable analysis, while maintaining backend control over database operations and access-control restrictions. However, the interpretation step remains a potential source of incorrect or manipulated query parameters, since deterministic backend execution does not guarantee the correctness or security of the extracted entities.

The current system also depends on the availability of the external Gemini API for AI-assisted queries. Temporary service failures or rate-limit responses may affect AI query processing. Future implementations could introduce additional fallback mechanisms or alternative models to improve availability and resilience.

Finally, the current evaluation primarily focuses on functional validation of the implemented features. A larger-scale evaluation involving multiple departments, teachers, and students could provide quantitative evidence regarding usability, response time, conflict-detection effectiveness, and system scalability.

Future development can therefore focus on expanding constraint validation, introducing optional timetable generation and optimization, improving conversational query capabilities, strengthening service resilience, and conducting broader empirical evaluation. These extensions would build upon the existing centralized architecture while preserving the system’s rule-validated and role-controlled design.

9. Conclusion

This paper presents an AI-assisted university timetable management system designed to address shared-faculty scheduling challenges and streamline administrative workflows. By combining centralized database management, role-based authorization, automated rule-based conflict validation, and a multi-level approval pipeline, the platform eliminates scheduling overlaps across departments. Furthermore, by employing Google Gemini 2.5 Flash strictly as an intent and entity extraction engine, the system delivers natural-language querying capabilities while maintaining strict deterministic control over underlying database operations and application logic.

References

Chen, M. C., Sze, S. N., Goh, S. L., Sabar, N. R., & Kendall, G. (2021). A survey of university course timetabling problem: Perspectives, trends and opportunities. *IEEE Access*, *9*, 106515–106529. https://doi.org/10.1109/ACCESS.2021.3100613

Dimopoulou, M., & Miliotis, P. A. (2004). An automated university course timetabling system developed in a distributed environment: A case study. *European Journal of Operational Research*, *153*(1), 136–147. https://doi.org/10.1016/S0377-2217(03)00104-8

Ferraiolo, D. F., Barkley, J. F., & Kuhn, D. R. (1999). A role-based access control model and reference implementation within a corporate intranet. *ACM Transactions on Information and System Security*, *2*(1), 34–64. https://doi.org/10.1145/300830.300834

Hong, Z., Yuan, Z., Zhang, Q., et al. (2025). Next-generation database interfaces: A survey of LLM-based text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering*, *37*(12), 7328–7345. https://doi.org/10.1109/TKDE.2025.3609486

Liu, X., Shen, S., Li, B., et al. (2025). A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*. https://doi.org/10.1109/TKDE.2025.3592032

Müller, T., Rudová, H., & Müllerová, Z. (2025). Real-world university course timetabling at the International Timetabling Competition 2019. *Journal of Scheduling*, *28*(2), 247–267. https://doi.org/10.1007/s10951-023-00801-w

Nourmohammadi-Khiarak, J., Zamani-Harghalani, Y., & Feizi-Derakhshi, M.-R. (2020). Combined multi-agent method to control inter-department common events collision

for university courses timetabling. *Journal of Intelligent Systems*, *29*(1), 110–126. https://doi.org/10.1515/jisys-2017-0249

Pedro, R., Coimbra, M. E., Castro, D., et al. (2025). Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*. https://doi.org/10.1109/ICSE55347.2025.00007

Quamar, A., Efthymiou, V., Lei, C., & Özcan, F. (2022). Natural language interfaces to data. *Foundations and Trends in Databases*, *11*(4), 319–414. https://doi.org/10.1561/1900000078

Zhu, X., Li, Q., Cui, L., & Liu, Y. (2024). Large language model enhanced text-to-SQL generation: A survey. *arXiv*. https://arxiv.org/abs/2410.06011